UNIVERSIDAD MAYOR
FACULTAD DE CIENCIAS, INGENIERÍA Y TECNOLOGÍA
Escuela de Ingeniería Civil en Computación e Informática

Ingeniería Civil en Computacion e Informática
Arquitectura de Desarrollo [móvil y web]
INGT1003-001

DriWin

Documento técnico del back end · Hito 2

Universidad Mayor
Profesor: Carlos Muñoz S.
Equipo: DriWin
Integrantes del Proyecto:
Acevedo, Vicente (21.626.029-5)
Cáceres, Javier (21.911.171-1)
Castillo, Joaquín (21.723.477-8)
Santiago, 30 de octubre de 2026
Facultad de Ciencias, Ingeniería y Tecnología

CONTENIDO

Índice

1	**Introducción**	3
2	**Cambios respecto del Hito 1**	4
2.1	De Rust a TypeScript en el servidor (C1, C2 y C3)	5
3	**Arquitectura del sistema y tecnologías**	6
3.1	Tecnologías elegidas y costo asumido	6
4	**Decisión: monolito o distribuido**	8
4.1	Carga esperada	8
4.2	Costo de operación	8
4.5	Cuándo revisaríamos la decisión	9
5	**Diseño de la API REST**	10
5.1	Convenciones generales	10
5.2	Tabla de recursos	10
5.3	Ejemplos de carga útil	11
5.4	Catálogo de códigos de estado	12
5.5	Criterio de versionado	13
6	**Separación de responsabilidades y capas**	14
6.1	Estructura de carpetas y capa de cada archivo	14
6.2	Recorrido completo de una petición	14
7	**Persistencia y manejo de credenciales**	16
7.1	Por qué PostgreSQL	16
7.2	Modelo de datos	16
7.3	Script de creación reproducible	17
7.4	Credenciales fuera del repositorio	17
8	**Despliegue en Azure, CI/CD y pruebas**	19
8.1	Servicio publicado	19
8.2	Integración y despliegue continuo con GitHub Actions	19
8.3	Pruebas	19
—	**Conclusión**	20
—	**Bibliografía**	21
—	**Anexos**	23
A	Declaración de uso de inteligencia artificial	23
B	Catálogo de los 9 drivers reales	23
C	Cómo levantar el servicio en otro equipo	23

PUNTO DE PARTIDA

1. Introducción

En el Hito 1, presentamos DriWin, una propuesta de mantenimiento, iniciación y limpieza de equipos basada en perfiles, esta dependía de un concepto que solo existía en el diagrama: un servicio en la nube capaz de recibir los identificadores de hardware de un PC (códigos PCI/USB VEN/DEV) junto al perfil del usuario y devolver la URL del controlador correcto.

A continuación se presentara ese motor (Con ayuda de una API REST en Node.js + Express 5 + TypeScript), ya construido, desplegado y funcionando, al igual que como evoluciono este motor a diferencia de primer hito, explicar el funcionamiento y estructura de nuestra base de datos (En Azure App Service, con persistencia en Azure Database for PostgreSQL y los instaladores almacenados en Azure Blob Storage), el contrato de la API, las capas y responsabilidades de cada una, valor de funcionamiento y el uso y protección de credenciales del programa.

TRAZABILIDAD

2. Cambios respecto del Hito 1

El Hito 1 fue una propuesta; el Hito 2 preparo la base del producto. Algunas decisiones se mantuvieron (perfiles de intención, arquitectura cliente-servidor en capas, datos en Azure) y otras se movieron al enfrentarlas con el tamaño del equipo y lo que aprendimos al programar. La Tabla 1 resume cada cambio con su motivo y su impacto; las subsecciones explican los más importantes.

Tabla 1. *Cambios respecto del Hito 1: qué se movió, por qué y qué impacto tuvo*

#	Qué cambió	Hito 1 (propuesta)	Hito 2 (construido)	Por qué	Impacto
C1	Lenguaje del back end	Rust (Opción 1: TypeScript + React + Rust)	TypeScript sobre Node.js 24 con Express 5	Rust exigía una curva de aprendizaje que no cabía en 6 semanas; Express es el contenido de la Unidad 2 y el equipo ya leía TypeScript	Un solo lenguaje en todo el sistema; tipos compartidos; 66 pruebas automáticas. Se pierde el acceso nativo de bajo nivel (ver C2)
C2	Lectura del hardware	Rust leyendo parámetros físicos de forma nativa	PowerShell + WMI/CIM (`Win32_PnPSignedDriver`) invocado desde Electron	Al salir Rust, la forma más directa y documentada de obtener VEN/DEV en Windows es WMI	Se obtienen los ID de todos los dispositivos en una llamada; no se leen temperaturas (fuera del alcance del MVP)
C3	Contenedor de escritorio	No definido (React + Rust sugería Tauri)	Electron 44 + electron-builder (instalador NSIS y portable)	Electron usa el mismo TypeScript/Node que el resto; Tauri requería Rust	Instalador más pesado (~100 MB), compensado por un único stack que todos pueden mantener
C4	Capas del servidor	3 capas conceptuales (presentación, negocio, datos)	5 capas en `server-api/src`: rutas, validación, controladores, servicios y repositorios	La rúbrica exige que el controlador no tenga lógica ni SQL; separarlo en archivos lo hace verificable	Cada petición recorre un camino fijo y explicable (sección 6)
C5	Infraestructura	"API alojada en Microsoft Azure" (sin detalle)	App Service F1 Linux (Canada Central), PostgreSQL Flexible Server B1ms y Blob Storage `driwinstorage`	Pasar de idea a servicio público con HTTPS dentro del crédito Azure for Students	API pública real; cold start de ~40 s por el plan gratuito (sección 8)
C6	Entrega de instaladores	La BD entrega "nombre y URL" del driver	La API entrega metadatos y una URL a Blob; el binario nunca pasa por la API (offloading)	Los instaladores pesan entre 9 y 945 MB; pasarlos por un plan F1 lo saturaría	Respuestas JSON de pocos KB; el ancho de banda pesado lo asume Blob
C7	Catálogo de drivers	Drivers de ejemplo con URLs de fabricante	9 drivers reales subidos a Blob (migración `002`)	Las URLs de ejemplo daban 404 y la firma NVIDIA (`DEV_2504`) no coincidía con el hardware real	Las 9 URLs responden 200; la NVIDIA se corrigió a `DEV_2684`
C8	Seguridad de red de la BD	No abordada	Firewall cerrado: se borró la regla `AllowAll` y quedaron 43 reglas con las IP de salida del App Service; TLS obligatorio	Una BD abierta a todo Internet contradice la Ley 21.719 citada en el Hito 1	Solo el App Service llega a la BD; administrar exige abrir una regla temporal
C9	Despliegue	No abordado	CI/CD con GitHub Actions: typecheck → pruebas → build → deploy	Desplegar a mano era lento y propenso a subir código roto	Un push a `main` que toca `server-api/` publica solo si pasan las 66 pruebas

#	Qué cambió	Hito 1 (propuesta)	Hito 2 (construido)	Por qué	Impacto
C10	Autenticación	No abordada	API Key en cabecera `X-API-Key` con dos roles: escritura y solo lectura	Las rutas de escritura no pueden quedar abiertas; la app de escritorio solo necesita leer	El .exe lleva solo la clave de lectura; un POST con ella recibe 403
C11	Usuarios	Cuentas de usuario y mejora a "DriWin Pro"	Usuarios sintéticos (6 en la semilla + 100 generados) y planes modelados en `subscription_tiers`; sin pagos	Minimización de datos (Ley 21.719) y alcance del semestre	No se guardan datos personales reales; el pago queda para una etapa posterior
C12	Acciones en el PC	Instalar drivers/apps y limpiar bloatware desde la app	La recomendación y la descarga son reales; la instalación silenciosa, la limpieza y el aviso UAC están **simulados** en la interfaz	Ejecutar instaladores con privilegios exige pruebas en equipos reales que aún no hicimos	Limitación declarada; los `silent_install_flags` de los 9 drivers no están verificados

Fuente: elaboración propia a partir del documento del Hito 1 y del historial del repositorio.

2.1 De Rust a TypeScript en el servidor (C1, C2 y C3)

En el Hito 1 elegimos "TypeScript (React) + CSS + Rust" y escribimos que Rust "va a ser una tarea complicada". Al planificar el Hito 2 confirmamos que lo era: el equipo tiene tres integrantes, ninguno había escrito Rust y el plazo era de unas seis semanas. Mantener Rust habría significado aprender un lenguaje nuevo justo cuando había que entregar la API, el modelo de datos y el despliegue. Por eso el servidor se escribió en **TypeScript sobre Node.js**, el mismo lenguaje del front end, y con **Express**, que es el framework que trabaja la Unidad 2 del curso.

El costo de ese cambio fue perder la lectura nativa de hardware que justificaba Rust. Lo resolvimos moviendo esa responsabilidad al cliente: la app de escritorio (Electron) ejecuta PowerShell con `Get-CimInstance Win32_PnPSignedDriver`, que entrega en una sola llamada el identificador de instancia de cada dispositivo (de donde se extraen VEN/DEV o VID/PID) y su driver instalado (Microsoft, s.f.-a). Como consecuencia, el contenedor de escritorio pasó a ser **Electron**, que comparte runtime con el servidor. El repositorio no contiene Rust ni Tauri.

VISIÓN GENERAL

3. Arquitectura del sistema y tecnologías

La Figura 1 muestra el sistema tal como está desplegado. El cliente es una app de escritorio para Windows; el servidor es un único proceso Node.js en Azure App Service, organizado en capas; los datos viven en PostgreSQL y los instaladores en Blob Storage. El cliente descarga los instaladores directamente desde Blob: la API solo decide **cuál** descargar.

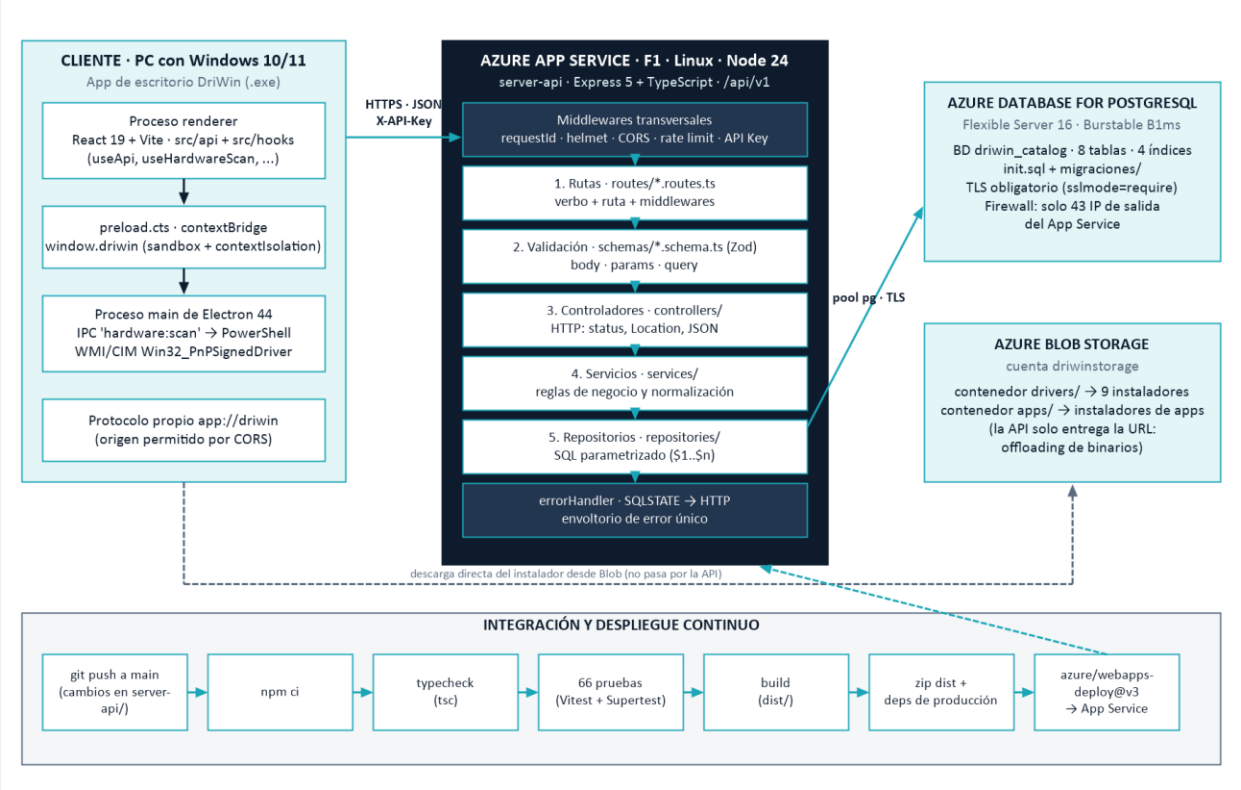


Figura 1. *Arquitectura desplegada de DriWin: cliente Electron, API en capas sobre Azure App Service, PostgreSQL, Blob Storage y pipeline de CI/CD.*

3.1 Tecnologías elegidas y costo asumido

Cada tecnología se eligió con criterios explícitos y dejando escrito qué costo aceptamos. La Tabla 2 resume el back end; la lista completa de dependencias del front end y la app de escritorio está en el `package.json` de cada carpeta.

Tabla 2. *Tecnologías del back end, alternativa considerada y costo asumido*

Pieza	Elección	Por qué (criterio)	Alternativa	Costo que asumimos
Lenguaje	TypeScript 5 sobre Node.js 24 LTS	Mismo lenguaje en cliente y servidor; tipos estáticos detectan errores de contrato al compilar	Rust	Node usa un solo hilo: no sirve para cálculo intensivo (DriWin no lo necesita)
Framework HTTP	Express 5	Middlewares componibles; Express 5 propaga solo los errores de funciones `async` (Express.js, s.f.-b)	Fastify, NestJS	Express no impone estructura: las capas son disciplina nuestra

Pieza	Elección	Por qué (criterio)	Alternativa	Costo que asumimos
Validación	Zod 3	Un esquema valida y además genera el tipo TypeScript; mensajes en español	Joi, validación manual	Esquemas duplican parte de las restricciones del SQL
Acceso a datos	`pg` (node-postgres) con SQL escrito a mano	Control total del SQL, consultas parametrizadas y transacciones (node-postgres, s.f.-b)	Prisma, TypeORM	Más código que un ORM; los tipos de fila se mantienen a mano
Motor de datos	PostgreSQL 16 (Azure Flexible Server)	Claves foráneas, restricciones UNIQUE/CHECK, arreglos `TEXT[]` e índices GIN	MySQL, Cosmos DB	Servidor administrado con costo mensual; requiere firewall y TLS
Seguridad HTTP	helmet 8, cors 2, express-rate-limit 7	Cabeceras seguras, lista blanca de orígenes y 60 peticiones/min por IP	API Management de Azure	El límite vive en memoria: se pierde si hubiera varias instancias
Hosting	Azure App Service F1 (Linux)	Gratis, HTTPS incluido y despliegue desde GitHub Actions	Máquina virtual, contenedores	Sin "Always On": el primer acceso tras inactividad tarda ~40 s
Binarios	Azure Blob Storage	Almacenamiento de objetos barato, servido por HTTPS sin pasar por la API (Microsoft, s.f.-g)	Servir archivos desde Express	Las URLs son públicas; un token SAS queda preparado en `AZURE_SAS_TOKEN`
Pruebas	Vitest 2 + Supertest 7	Pruebas de contrato HTTP sin levantar el puerto	Jest	—

DECISIÓN DE ARQUITECTURA

4. Decisión: monolito o distribuido

> **Postura del equipo: DriWin usa un monolito modular en capas**, desplegado como una sola unidad en Azure App Service y apoyado en servicios administrados (PostgreSQL y Blob Storage). El sistema completo es cliente-servidor y ya está repartido en la nube, pero la lógica del servidor vive en un único proceso. No usamos microservicios.

4.1 Carga esperada

DriWin no es una red social (No espera tanto flujo de usuarios) : un usuario abre la app al configurar un PC nuevo o cuando quiere revisar sus drivers, no todos los días. La Tabla 3 estima la carga con supuestos explícitos.

Tabla 3. *Estimación de carga de la API (supuestos declarados)*

Variable	Valor	Fuente o supuesto
Usuarios del piloto	100 usuarios sintéticos cargados; meta de 500 equipos el próximo semestre	Script `npm run seed:synthetic`; meta propia
Peticiones por sesión	10 a 20 (health, apps por perfil y una recomendación por dispositivo relevante: GPU, Wi-Fi, Bluetooth)	Flujo real de la app de escritorio
Sesiones por equipo	2-3 al mes	Supuesto: instalación inicial + una revisión
Volumen mensual	500 × 2 × 20 = 20.000 peticiones (≈ 0,008 por segundo de promedio)	Cálculo
Pico realista	Un técnico (persona "Bilberto" del Hito 1) configurando 20 PC en una hora: ≈ 400 peticiones/h ≈ 0,1 por segundo	Supuesto
Tamaño de respuesta	Pocos KB de JSON; los instaladores (9 a 945 MB) van directo desde Blob	Offloading (C6)
Latencia medida	`/health` con consulta a la BD: ~220 ms con la instancia activa	Medición del 30-10-2026

Incluso multiplicando el pico por 100, la carga queda muy por debajo de lo que maneja un solo proceso Node.js. Además, la parte pesada del tráfico (los gigabytes de instaladores) ya está fuera de la API gracias a Blob Storage.

4.2 Costo de operación

Tabla 4. *Costo mensual aproximado: arquitectura actual frente a una versión distribuida*

Componente	Monolito actual	Distribuido (3 servicios: catálogo, recomendaciones, bloatware)
Cómputo de la API	App Service F1: USD 0 (60 min de CPU/día, 1 GB de RAM)	3 apps; para comunicarse entre sí sin cold start en cadena necesitarían un plan pagado con "Always On"
Base de datos	PostgreSQL Flexible B1ms: aprox. USD 15–20 (cómputo + 32 GB)	Una BD por servicio (patrón recomendado) triplica este ítem, o se comparte y se pierde la independencia

Componente	Monolito actual	Distribuido (3 servicios: catálogo, recomendaciones, bloatware)
Almacenamiento de binarios	Blob Storage: centavos (≈ 4,7 GB en `drivers/`)	Igual
Puerta de entrada	No se necesita	API gateway o enrutamiento propio
Crédito consumido	23 USD en 13 días antes de bajar la BD de B2s a B1ms	Agotaría el crédito Azure for Students antes del Hito 3

Precios de lista referenciales de Microsoft (s.f.-h; s.f.-i); varían por región y tipo de cambio. El plan F1 no tiene costo y sus límites figuran en Microsoft (s.f.-b).

4.5 Cuándo revisaríamos la decisión

El monolito está preparado para crecer: la API no guarda estado de sesión (cada petición trae su credencial), así que puede ejecutarse en varias instancias. Revisaremos la decisión, en este orden, si se da alguno de estos escenarios:

1. **Se agota la cuota de CPU del F1 o el cold start afecta a usuarios reales.** Primera respuesta: escalar verticalmente (plan con Always On), sin cambiar la arquitectura.
2. **La latencia p95 supera 500 ms de forma sostenida con más de 50 peticiones por segundo.** Escalar horizontalmente (varias instancias). Antes habría que mover el contador de `express-rate-limit` a un almacén compartido (por ejemplo, Redis), porque hoy vive en memoria.
3. **Aparece un módulo con un perfil de carga o de riesgo distinto**, por ejemplo telemetría de instalaciones (muchas escrituras) o pagos de "DriWin Pro" (requisitos de cumplimiento). Ese módulo sería el primer candidato para separarse como servicio propio.

CONTRATO

5. Diseño de la API REST

La API está orientada a recursos, usa los verbos HTTP según su semántica (Fielding et al., 2022) e intercambia JSON. El contrato se escribió antes de programar y está implementado en `server-api/src/routes` y `server-api/src/schemas`; la colección de Postman `docs/postman/DriWin_Hito2.postman_collection.json` lo ejecuta contra Azure.

5.1 Convenciones generales

Tabla 5. *Convenciones transversales del contrato*

Aspecto	Regla
URL base	`https://driwin-api-hsgshffkeydpf2fe.canadacentral-01.azurewebsites.net/api/v1`
Formato	JSON UTF-8. Cuerpo máximo de 100 KB (si se excede: 413). Recursos en plural y kebab-case (`/driver-packages`).
Autenticación	Cabecera `X-API-Key` (o `Authorization: Bearer`). Rol **read-write** para POST, PUT y DELETE; rol **read-only** para la app de escritorio. Comparación en tiempo constante (`crypto.timingSafeEqual`).
Lecturas de catálogo	Los GET de catálogo admiten acceso sin clave. Si se envía una clave inválida se responde 401. Las rutas `/recommendations` exigen clave.
Paginación	`?page=1&limit=20&order=asc&sort=<columna>`; `limit` máximo 100. Respuesta: `{ data: [...], meta: { page, limit, total, totalPages } }`.
Creación	`201 Created` con cabecera `Location` hacia el nuevo recurso y el objeto en `data`.
Borrado	`204 No Content` sin cuerpo.
Límite de tasa	60 peticiones por minuto por IP; al excederlo, 429 y cabeceras `RateLimit` (RFC 6585).
Trazabilidad	Cada respuesta lleva `X-Request-Id`; los errores incluyen el mismo `requestId` en el cuerpo.

5.2 Tabla de recursos

La Tabla 6 lista las 18 operaciones. Los códigos de error que pueden aparecer en cualquier ruta (400 por JSON mal formado, 401 por clave inválida, 413, 429, 500 y 503) se detallan en la sección 5.4 y no se repiten en cada fila.

Tabla 6. *Recursos, verbos, parámetros y códigos de estado de la API v1*

Verbo y ruta	Parámetros y cuerpo	Clave	Éxito	Errores propios
GET /health	—	No	200	503 BD caída
GET /hardware-components	query: `page`, `limit`, `order`, `sort`, `ven_id`, `dev_id`, `category`	Opcional	200	400 filtro inválido
GET /hardware-components/{id}	path: `id` entero > 0	Opcional	200	400, 404
POST /hardware-components	body: `ven_id`, `dev_id` (4 hex, acepta `VEN_/0x`), `subsys_id`?, `commercial_name`, `category`	Escritura	201 + Location	400, 401, 403, 409 firma duplicada
PUT /hardware-components/{id}	body: `commercial_name`? y/o `category`? (al menos uno)	Escritura	200	400, 401, 403, 404

Verbo y ruta	Parámetros y cuerpo	Clave	Éxito	Errores propios
DELETE /hardware-components/{id}	query: `cascade=true|false`	Escritura	204	401, 403, 404, 409 tiene drivers y falta `cascade=true`
GET /driver-packages	query: paginación, `component_id`, `profile_id`, `os_environment`, `whql`, `sort`	Opcional	200	400
GET /driver-packages/{id}	path: `id`	Opcional	200	400, 404
POST /driver-packages	body: `component_id`, `profile_id`, `os_environment`, `edition_name`, `version_number`, `release_date` (YYYY-MM-DD), `download_url`, `file_size_mb`, `silent_install_flags`?, `is_whql_certified`?	Escritura	201 + Location	400, 401, 403, 409 combinación duplicada, 422 FK inexistente
PUT /driver-packages/{id}	body: cualquier campo del POST (al menos uno)	Escritura	200	400, 401, 403, 404, 409, 422
DELETE /driver-packages/{id}	path: `id`	Escritura	204	401, 403, 404
GET /bloatware-rules	query: `risk_level=BAJO|MEDIO|ALTO`	Opcional	200	400
GET /bloatware-rules/{id}	path: `id`	Opcional	200	400, 404
POST /bloatware-rules	body: `app_display_name`, `package_signature`, `detection_powershell_cmd`, `removal_powershell_cmd`, `risk_level`, `description`	Escritura	201 + Location	400, 401, 403, 409 firma duplicada
PUT /bloatware-rules/{id}	body: cualquier campo del POST (al menos uno)	Escritura	200	400, 401, 403, 404, 409
DELETE /bloatware-rules/{id}	path: `id`	Escritura	204	401, 403, 404
POST /recommendations/driver	body: `ven_id`, `dev_id`, `subsys_id`?, `os_environment` (`Win11_x64|Win10_x64`), `profiles[]` (1 a 10)	Cualquier rol	200	400, 401, 404 hardware no catalogado, 409 perfil inexistente, 422 sin driver para el perfil
GET /recommendations/apps	query: `profiles=Gamer,Creator` (1 a 10, sin repetidos)	Cualquier rol	200	400, 401

Una decisión de diseño que conviene explicar: `POST /recommendations/driver` usa POST aunque **no escribe** en la base de datos. Se eligió así porque el cliente envía un objeto con el hardware detectado y una lista de perfiles, que no cabe de forma limpia en una URL. Por eso basta la clave de solo lectura.

5.3 Ejemplos de carga útil

Crear un componente de hardware

```
POST /api/v1/hardware-components
X-API-Key: <clave de escritura>
Content-Type: application/json

{ "ven_id": "VEN_10EC", "dev_id": "8168",
  "commercial_name": "Realtek PCIe GbE Family Controller", "category": "Network" }

HTTP/1.1 201 Created
Location: /api/v1/hardware-components/7
{ "data": { "component_id": 7, "ven_id": "10EC", "dev_id": "8168",
            "subsys_id": "GENERIC",
            "commercial_name": "Realtek PCIe GbE Family Controller",
            "category": "Network", "created_at": "2026-10-30T21:40:00.000Z" } }
```

El servicio normaliza `VEN_10EC` a `10EC` y asigna `subsys_id = "GENERIC"` cuando no se envía. Si se repite la misma firma, la restricción `uq_hardware_signature` produce un 409.

Pedir la recomendación de driver (caso de uso principal)

```
POST /api/v1/recommendations/driver
X-API-Key: <clave de solo lectura>

{ "ven_id": "10DE", "dev_id": "2684",
  "os_environment": "Win11_x64", "profiles": ["Gamer", "Creator"] }

HTTP/1.1 200 OK
{ "data": {
    "component": { "component_id": 2, "ven_id": "10DE", "dev_id": "2684",
      "commercial_name": "NVIDIA GeForce RTX 30/40 Series", "category": "GPU" },
    "recommendation": { "driver_id": 2, "profile_name": "Gamer",
      "edition_name": "NVIDIA GeForce Game Ready Package Launcher",
      "version_number": "561.09", "file_size_mb": 945,
      "is_whql_certified": true,
      "download_url": "https://driwinstorage.blob.core.windows.net/drivers/...",
      ... },
    "alternatives": [ { "profile_name": "Creator", ... } ],
    "offloading": { "strategy": "DIRECT_AZURE_BLOB", "container": "drivers",
      "estimated_bytes": 990904320, "binary_traverses_api": false } },
  "meta": { "generated_at": "2026-10-30T21:41:02.114Z", "query_time_ms": 38 } }
```

Respuesta abreviada con "..." en los campos que no aportan al ejemplo.

Respuesta de error (mismo envoltorio en toda la API)

```
HTTP/1.1 422 Unprocessable Entity
{ "error": {
    "code": "NO_DRIVER_FOR_PROFILE",
    "message": "No existe un paquete de controladores para VEN_8086 / DEV_A780
                en Win11_x64 para el perfil solicitado.",
    "details": [ { "field": "profiles",
                   "issue": "No hay driver publicado para Gamer en Win11_x64." } ],
    "timestamp": "2026-10-30T21:42:10.551Z",
    "path": "/api/v1/recommendations/driver",
    "requestId": "req_95c23c45945a4cf98593",
    "hint": "Pruebe otro perfil u otro sistema operativo; el hardware sí está
             catalogado." } }
```

5.4 Catálogo de códigos de estado

Los errores se generan en un solo lugar (`middlewares/errorHandler.ts`). Los errores de PostgreSQL se traducen por su código SQLSTATE (PostgreSQL Global Development Group, s.f.-a), de modo que una violación de integridad nunca llega al cliente como un 500 genérico.

Tabla 7. *Códigos de estado y su origen*

HTTP	Código interno	Cuando ocurre
200 / 201 / 204	—	Lectura o actualización correcta / recurso creado (con `Location`) / borrado
400	`VALIDATION_ERROR`, `MALFORMED_JSON`	Falla un esquema Zod (con detalle por campo), JSON mal formado, o SQLSTATE `23514`, `23502`, `22P02`, `22001`
401	`UNAUTHORIZED`	Falta la clave en una ruta que la exige, o la clave enviada no es válida
403	`FORBIDDEN`	Clave de solo lectura en una operación de escritura, u origen no autorizado por CORS

HTTP	Código interno	Cuando ocurre
404	`NOT_FOUND`	El id no existe, la firma de hardware no está catalogada o la ruta no existe
409	`CONFLICT`	SQLSTATE `23505` (duplicado), perfil inexistente o borrado sin `cascade=true`
413	`PAYLOAD_TOO_LARGE`	Cuerpo mayor a 100 KB
422	`FK_VIOLATION`, `NO_DRIVER_FOR_PROFILE`	SQLSTATE `23503` (referencia inexistente) o hardware y perfil válidos sin paquete publicado
429	`RATE_LIMITED`	Más de 60 peticiones por minuto desde la misma IP
500	`INTERNAL_ERROR`	Error no previsto; el log guarda método, ruta y `requestId`, nunca el cuerpo ni las cabeceras
503	`DB_UNAVAILABLE`	PostgreSQL no responde (SQLSTATE `08xxx`, `53300`, `57014`, `ECONNREFUSED`, `ETIMEDOUT`)

5.5 Criterio de versionado

La versión mayor forma parte de la URL (`/api/v1`, configurable con `API_BASE_PATH`) y el número SemVer completo (`1.0.0`, en `config/version.ts`) se informa en `/health`. Lo que decide si un cambio se queda en `v1` o abre una nueva versión es si rompe o no a los clientes que ya están instalados:

Cuándo se mantiene v1 y cuándo se crea v2

Tipo de cambio	Ejemplos	Qué hacemos	Estado
Compatible	Agregar un endpoint, un campo opcional en la entrada o un campo nuevo en la respuesta	Se queda en /api/v1 y sube la versión menor (1.0.0 → 1.1.0)	Aplicado (ver ejemplo)
Incompatible	Renombrar o quitar un campo, cambiar un tipo o el significado de un código de estado	Se publica /api/v2 y v1 sigue activa en paralelo durante la transición	Regla para el futuro: aún no ha ocurrido

Ejemplo real (compatible). En el Hito 1 la petición de recomendación enviaba `ven`, `dev` y `profile`. En el Hito 2 se adoptaron los nombres de las columnas de la base (`ven_id`, `dev_id`) y una lista `profiles[]` para combinar perfiles (por ejemplo, Gamer + Creator). En lugar de romper el contrato, `recommendation.schema.ts` acepta ambos nombres y los convierte a la forma nueva, por eso el cambio se quedó en `v1`. La excepción es `os`, que pasó a llamarse `os_environment` sin alias: agregarlo es el próximo cambio compatible pendiente.

Por qué v1 no se apaga de inmediato. La app de escritorio no se actualiza sola, así que un `.exe` ya instalado seguirá llamando a `/api/v1`. Si algún día se publica `/api/v2`, las respuestas de `v1` incluirán la cabecera `Sunset` con la fecha de retiro (Wilde, 2019), para que los clientes antiguos alcancen a actualizarse. Hoy no existe `/api/v2`.

SEPARACIÓN DE RESPONSABILIDADES

6. Separación de responsabilidades y capas

Nuestra regla del proyecto es simple y se puede revisar abriendo los archivos: las rutas no contienen lógica, los controladores solo hablan HTTP, los servicios contienen las reglas de negocio y únicamente los repositorios escriben SQL. La validación de entrada ocurre antes de llegar al controlador.

6.1 Estructura de carpetas y capa de cada archivo

```
server-api/src/
├── server.ts       # arranque y cierre ordenado
├── app.ts          # middlewares globales + /api/v1
├── config/         # CONFIGURACIÓN (env, db, version)
├── middlewares/    # TRANSVERSAL (auth, validate, errores)
├── routes/         # CAPA 1 · RUTEO
├── schemas/        # CAPA 2 · VALIDACIÓN (Zod)
├── controllers/    # CAPA 3 · CONTROLADORES
├── services/       # CAPA 4 · NEGOCIO
├── repositories/   # CAPA 5 · ACCESO A DATOS (SQL)
├── domain/         # MODELO (tipos, ApiError, paginación)
└── scripts/        # HERRAMIENTAS (seed sintético)
```

Tabla 8. *Responsabilidad y regla de cada capa*

Capa	Archivos de ejemplo	Responsabilidad	Lo que tiene prohibido
Ruteo	`routes/hardware.routes.ts`	Declarar verbo, ruta y cadena de middlewares	SQL y reglas de negocio
Validación	`schemas/recommendation.schema.ts`	Validar y normalizar `body`, `params` y `query` con Zod	Acceder a la BD
Controlador	`controllers/hardware.controller.ts`	Leer el DTO validado, llamar al servicio y elegir código, cabeceras y JSON	Lógica de negocio y SQL
Servicio	`services/recommendation.service.ts`	Reglas: normalizar firmas PCI, deduplicar perfiles, decidir 404/409/422, mapear a DTO	Conocer `req` o `res`; escribir SQL
Repositorio	`repositories/recommendation.repository.ts`	Única fuente de SQL: columnas explícitas, parámetros `$n`, transacciones	Decidir códigos HTTP
Transversal	`middlewares/errorHandler.ts`	Traducir errores a un envoltorio único	—

6.2 Recorrido completo de una petición

Usando de ejemplo un caso propuesto en el hito 1: Un usuario con perfil Gamer abre DriWin en un PC con una GPU NVIDIA RTX. La Tabla 9 sigue la petición desde el escaneo hasta la base de datos y de vuelta.

Tabla 9. *Recorrido de POST /api/v1/recommendations/driver, capa por capa*

Paso	Dónde (archivo)	Qué ocurre
1	`electron/hardwareScanner.ts` y `electron/main.ts`	El proceso main de Electron atiende el IPC `hardware:scan` y ejecuta PowerShell con `Get-CimInstance Win32_PnPSignedDriver`. Extrae `VEN_10DE` y `DEV_2684` del identificador de instancia.
2	`electron/preload.cts`	El resultado llega a React por `window.driwin`, expuesto con `contextBridge`; la interfaz no tiene acceso directo a Node (Electron, s.f.-a).
3	`src/hooks` y `src/api`	El hook de recomendaciones arma el cuerpo y hace `fetch` con la clave de solo lectura; un `AbortController` cancela peticiones obsoletas.
4	`app.ts`	En Azure, la petición pasa por `requestId`, `helmet`, `cors` (el origen `app://driwin` está permitido), el logger y `express.json` con límite de 100 KB.
5	`middlewares/rateLimit.ts`, `authenticate.ts`	Se descuenta del límite por IP y la clave se compara en tiempo constante: `req.apiRole = "read-only"`.
6	`routes/index.ts` → `recommendations.routes.ts`	Se ejecuta `requireApiKey` y luego `validate({ body: driverRecommendationBodySchema })`, que valida los campos y convierte los alias del Hito 1 a la forma canónica.
7	`controllers/recommendations.controller.ts`	`postDriverRecommendation` obtiene el DTO validado, mide el tiempo y llama al servicio. No decide nada de negocio.
8	`services/recommendation.service.ts`	`resolveOptimalDriver` normaliza las firmas a 4 hexadecimales en mayúscula, elimina perfiles repetidos y llama al repositorio. Si no hay filas, distingue: hardware desconocido (404), perfil inexistente (409) o sin driver para ese perfil (422).
9	`repositories/recommendation.repository.ts`	Ejecuta un `SELECT` parametrizado que une `driver_packages`, `hardware_components` y `user_profiles`, filtra por `ven_id`, `dev_id`, sistema operativo y `profile_name = ANY($4)`, y ordena por WHQL y tamaño. Usa el índice `idx_hardware_lookup`.
10	`config/db.ts` → PostgreSQL	La consulta sale por el pool de `pg` con TLS hacia Azure; el firewall solo acepta las IP del App Service.
11	Vuelta: servicio → controlador	El servicio arma el DTO (recomendación, alternativas y descriptor de offloading); el controlador responde `200` con `data` y `meta`. Ante cualquier error, `errorHandler` responde con el envoltorio de la sección 5.3.
12	Cliente → Blob Storage	La interfaz muestra el driver; al presionar "Instalar" el archivo se descarga directamente desde `driwinstorage.blob.core.windows.net`, sin pasar por la API.

El orden de responsabilidades se puede comprobar en el código: el único `SELECT` del recorrido está en el repositorio, el único `res.status(...)` está en el controlador y las decisiones 404/409/422 están en el servicio.

DATOS

7. Persistencia y manejo de credenciales

7.1 Por qué PostgreSQL

El dominio de DriWin es relacional: un driver pertenece a un componente y a un perfil, y la misma firma de hardware no puede repetirse. PostgreSQL permite expresar esas reglas en el motor (claves foráneas, `UNIQUE`, `CHECK`) para que ninguna ruta de la API pueda dejar datos inconsistentes (PostgreSQL Global Development Group, s.f.-b). Además, ofrece arreglos `TEXT[]` con índice GIN (perfiles elegidos por usuario) y transacciones con bloqueo de fila (`SELECT ... FOR UPDATE`), que usamos para el borrado protegido de componentes. El costo asumido es operar un servidor administrado con costo mensual, firewall y TLS (Tráfico cifrado) obligatorios.

7.2 Modelo de datos

La base `driwin_catalog` tiene 8 tablas y 4 índices secundarios (Figura 2). El diagrama Mermaid equivalente está en `db/modelo_datos.md`.

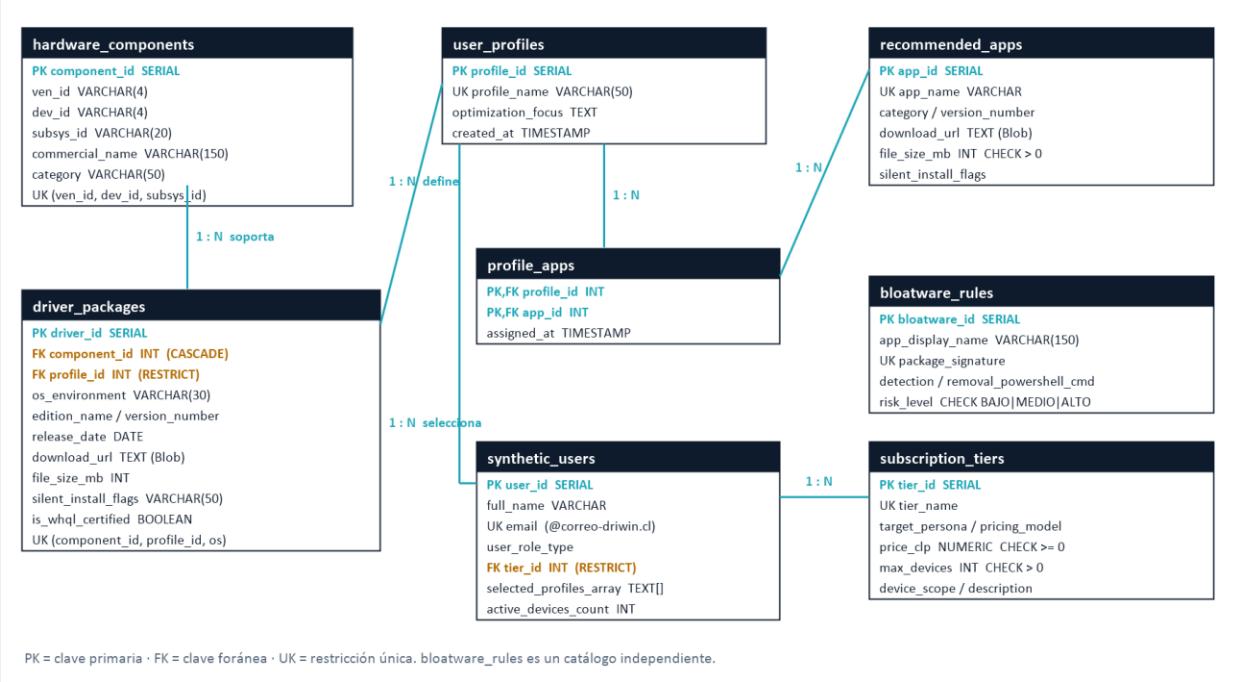


Figura 2. *Modelo entidad-relación de driwin_catalog con claves primarias, foráneas y restricciones únicas.*

Tabla 10. *Diccionario resumido de tablas*

Tabla	Clave primaria	Relaciones (FK)	Restricciones e índices	Propósito
`user_profiles`	`profile_id` SERIAL	—	`profile_name` UNIQUE	Perfiles Gamer, Creator, Office y Developer
`hardware_components`	`component_id` SERIAL	—	UNIQUE (`ven_id`, `dev_id`, `subsys_id`); índice `idx_hardware_lookup`	Firmas PCI/USB con nombre comercial

Tabla	Clave primaria	Relaciones (FK)	Restricciones e índices	Propósito
`driver_packages`	`driver_id` SERIAL	`component_id` (ON DELETE CASCADE), `profile_id` (RESTRICT)	UNIQUE (componente, perfil, SO); índice por perfil	Driver recomendado y su URL en Blob
`recommended_apps`	`app_id` SERIAL	—	`app_name` UNIQUE; CHECK `file_size_mb > 0`	Catálogo de aplicaciones
`profile_apps`	(`profile_id`, `app_id`)	Ambas columnas	PK compuesta; índice por `app_id`	Tabla puente N:M perfil ↔ app
`bloatware_rules`	`bloatware_id` SERIAL	—	`package_signature` UNIQUE; CHECK riesgo BAJO, MEDIO o ALTO	Detección y retiro de bloatware
`subscription_tiers`	`tier_id` SERIAL	—	CHECK precio ≥ 0 y dispositivos > 0	Planes comerciales (DriWin Pro)
`synthetic_users`	`user_id` SERIAL	`tier_id` (RESTRICT)	`email` UNIQUE; CHECK al menos un perfil; índice GIN	Usuarios ficticios para pruebas

- **SERIAL:** Asigna automáticamente un número entero consecutivo único a cada nueva fila que se inserta en una tabla
- **UNIQUE:** Garantiza que todos los valores almacenados en esa columna sean únicos
- **CHECK:** Regla de validación lógica que deben cumplir los datos antes de ser guardados
- **PK / Primary Key / Clave Primaria:** Identificador principal y único de cada registro en una tabla
- **FK / Foreign Key / Clave Foránea:** Un campo que conecta lógicamente una tabla con la clave primaria de otra tabla
- **GIN:** Evita que el motor de la base de datos deba recorrer toda la tabla fila por fila para encontrar un dato, mejorando drásticamente el rendimiento de las consultas

7.3 Script de creación reproducible

En el directorio `db/init.sql` crea la base desde cero: borra y recrea las 8 tablas, crea los índices y carga una semilla (4 perfiles, 6 componentes, 9 drivers, 20 apps, 20 vínculos perfil-app, 3 reglas de bloatware, 5 planes y 6 usuarios ficticios). Es idempotente: ejecutarlo dos veces deja el mismo resultado. Los cambios sobre una base ya existente se entregan como migraciones numeradas en `db/migrations/` (la `002` cargó los drivers reales y ajustó las secuencias `SERIAL`). Los 100 usuarios sintéticos adicionales se generan con `npm run seed:synthetic`, usando el dominio ficticio `@correo-driwin.cl` en cumplimiento de la minimización de datos de la Ley 21.719 (Ley N.º 21.719, 2024).

```
$DB = "<cadena de conexión>"
psql $DB -v ON_ERROR_STOP=1 -f db/init.sql
psql $DB -v ON_ERROR_STOP=1 -f db/migrations/002_catalogo_drivers_reales.sql
```

7.4 Credenciales fuera del repositorio

Ninguna credencial está escrita en el código. `git ls-files` solo muestra las plantillas `.env.example` (raíz, `server-api/` y `app-inicio-windows/`), y los `.gitignore` excluyen `.env`, `.env.*` y `*.env`, permitiendo solo las plantillas. La Tabla 11 indica dónde vive cada secreto.

Tabla 11. *Ubicación de cada credencial*

Credencial	Dónde vive	Quién la usa
`DATABASE_URL` (usuario y contraseña de PostgreSQL)	Variables de entorno del App Service en Azure; `.env` local de cada integrante (ignorado por git)	Solo el servidor
`API_KEY` (rol escritura)	Variables del App Service y `.env` local; colección de Postman con variable de entorno, sin valor versionado	Equipo, para la demo y la administración
`API_KEY_READ_ONLY`	Variables del App Service; `app-inicio-windows/.env.production` (ignorado) al compilar el .exe	App de escritorio; no permite escribir
`AZURE_WEBAPP_PUBLISH_PROFILE`	Secreto cifrado de GitHub Actions (GitHub, s.f.-b)	Solo el pipeline de despliegue
`AZURE_SAS_TOKEN` (opcional)	Variable del App Service	Reservado por si los contenedores pasan a privados

Además, `config/env.ts` valida todas las variables con Zod al arrancar y **detiene el servidor** si en producción la `API_KEY` conserva el valor de plantilla, tiene menos de 32 caracteres o la `DATABASE_URL` no incluye `sslmode=require`. Los mensajes de error nombran la variable pero nunca imprimen su valor. Para configurar el entorno local basta con copiar la plantilla:

```
Copy-Item server-api\.env.example server-api\.env   # luego completar DATABASE_URL y API_KEY
```

> **Pendiente declarado.** El servidor se conecta hoy con el usuario administrador `driwin_admin`. La plantilla ya usa un usuario `driwin_api`: el siguiente paso es crear ese rol con permisos solo de `SELECT`, `INSERT`, `UPDATE` y `DELETE` sobre las tablas del catálogo (mínimo privilegio) y rotar la contraseña actual.

OPERACIÓN

8. Despliegue en Azure, CI/CD y pruebas

8.1 Servicio publicado

Tabla 12. *Recursos de Azure en uso (grupo rg-driwin-hito2)*

Recurso	Configuración	Observación
Azure App Service `driwin-api`	Plan F1, Linux, Node 24, región Canada Central, HTTPS	Gratis; sin Always On, por lo que la primera petición tras inactividad tarda ~40 s
Azure Database for PostgreSQL	Flexible Server 16, Burstable B1ms, TLS obligatorio	Bajado desde B2s para cuidar el crédito
Firewall de la BD	43 reglas `driwin-api-1..43` = `possibleOutboundIpAddresses` del App Service	Si cambia el plan del App Service hay que recalcular las IP
Azure Blob Storage `driwinstorage`	Contenedores `drivers/` (9 instaladores, ≈ 4,7 GB) y `apps/`	Los instaladores de `apps/` aún no están cargados

8.2 Integración y despliegue continuo con GitHub Actions

El archivo `.github/workflows/deploy-backend.yml` se ejecuta en cada push a `main` que modifique `server-api/` (o a mano desde la pestaña Actions). Instala Node 24 con las dependencias exactas del lockfile (`npm ci`), verifica tipos, ejecuta las 66 pruebas, compila a `dist/`, elimina las dependencias de desarrollo y publica el zip con la acción oficial `azure/webapps-deploy@v3` (Microsoft, s.f.-c). Si una prueba falla, no se publica y Azure sigue con la versión anterior. El workflow tiene permisos mínimos (`contents: read`) y no se ejecutan dos despliegues a la vez (`concurrency`).

8.3 Pruebas

- **66 pruebas automáticas hechas con npm test** con Vitest y Supertest en `server-api/tests`: contrato HTTP de cada recurso, validaciones, traducción de errores SQLSTATE, reglas de bloatware, recomendaciones, `/health` y el generador de usuarios sintéticos.
- **Colección de Postman** (`docs/postman/DriWin_Hito2.postman_collection.json`) con el entorno `DriWin_Azure`: son las peticiones de la demostración en vivo (crear, leer, actualizar y borrar, mostrando el dato guardado).
- **Hardware simulado** en el front (`src/scanner/fixtures.ts`) para probar la interfaz sin depender del PC donde se presenta.

CIERRE

Conclusión

Qué se logró. La promesa del Hito 1 fue reestructurada y adaptada para construir los cimientos de la aplicación y que sea consistente a lo largo del tiempo. DriWin cuenta con una API REST versionada y publicada en Azure, cinco recursos con 18 operaciones (tres de ellos con CRUD completo), persistencia en PostgreSQL con un modelo de 8 tablas que hace cumplir sus reglas en el propio motor, credenciales fuera del repositorio, una base de datos cerrada al resto de Internet y un pipeline que aprueba y despliega el cambio en código siempre y cuando este haya sido probado, para subirlo por ejemplo a Azure donde se ubica la base de datos. La app de escritorio ya escanea el hardware real del PC y recibe recomendaciones de 9 drivers reales alojados en Blob Storage.

Qué quedó pendiente. La instalación silenciosa, la limpieza de bloatware y el aviso de permisos de administrador siguen simulados en la interfaz, y los parámetros silenciosos de los 9 instaladores no están verificados en equipos reales. Faltan los instaladores del contenedor `apps/`, el usuario de base de datos con mínimo privilegio. Estos puntos son el trabajo del Hito 3.

Qué aprendimos. Pudimos evidenciar que el costo de la seguridad, como también la comodidad que presenta el ir por alternativas más interactivas que ofrecen empresas como Azure, que, si bien nuestro sesgo a seleccionarla fue la suscripción gratuita para estudiantes a nuestra disposición, nos hizo replantearnos que servidores basados en Linux podrían resultar mucho más baratos y convenientes para pruebas y eficiencia de recursos.

Este proyecto si se llevara a una escala mayor en un futuro, se debería reestructurar la propuesta de distribución de drivers por problemas de licencias y distribución de material de terceros, lo que nos llevó a plantear el uso un repertorio acotado por la sostenibilidad de los métodos existentes de obtención de drivers.

Junto a TypeScript se logró diseñar el Back-End, junto a esto se aprendió de los procesos que se hacen para vincular tanto el Back-End con el Front-End, el hecho de colocar las carpetas por separadas y dejar un mejor orden nos ayudó tener un mejor orden dentro del proyecto/Repositorio, conectar la base de datos con el Back-End fue un desafió, pero se logró implementarlo sin problemas.

FUENTES

Bibliografía

Electron. (s.f.-a). *Context isolation*. Recuperado el 30 de octubre de 2026, de https://www.electronjs.org/docs/latest/tutorial/context-isolation

Electron. (s.f.-b). *Security*. Recuperado el 30 de octubre de 2026, de https://www.electronjs.org/docs/latest/tutorial/security

Express.js. (s.f.-a). *Production best practices: Security*. Recuperado el 30 de octubre de 2026, de https://expressjs.com/en/advanced/best-practice-security.html

Express.js. (s.f.-b). *Error handling*. Recuperado el 30 de octubre de 2026, de https://expressjs.com/en/guide/error-handling.html

Express.js. (s.f.-c). *Express 5.x API reference*. Recuperado el 30 de octubre de 2026, de https://expressjs.com/en/5x/api.html

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9110

Fowler, M. (2015, 3 de junio). *MonolithFirst*. martinfowler.com. https://martinfowler.com/bliki/MonolithFirst.html

GitHub. (s.f.-a). *Workflow syntax for GitHub Actions*. GitHub Docs. Recuperado el 30 de octubre de 2026, de https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions

GitHub. (s.f.-b). *Using secrets in GitHub Actions*. GitHub Docs. Recuperado el 30 de octubre de 2026, de https://docs.github.com/en/actions/security-for-github-actions/security-guides/using-secrets-in-github-actions

Ley N.º 21.719. (2024, 13 de diciembre). *Regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales*. Diario Oficial de la República de Chile. https://www.bcn.cl/leychile/navegar?idNorma=1209272

Microsoft. (s.f.-a). *Get-CimInstance (CimCmdlets)*. Microsoft Learn. Recuperado el 30 de octubre de 2026, de https://learn.microsoft.com/en-us/powershell/module/cimcmdlets/get-ciminstance

Microsoft. (s.f.-b). *Azure subscription and service limits, quotas, and constraints*. Microsoft Learn. Recuperado el 30 de octubre de 2026, de https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/azure-subscription-service-limits

Microsoft. (s.f.-c). *Deploy to App Service using GitHub Actions*. Microsoft Learn. Recuperado el 30 de octubre de 2026, de https://learn.microsoft.com/en-us/azure/app-service/deploy-github-actions

Microsoft. (s.f.-d). *Inbound and outbound IP addresses in Azure App Service*. Microsoft Learn. Recuperado el 30 de octubre de 2026, de https://learn.microsoft.com/en-us/azure/app-service/overview-inbound-outbound-ips

Microsoft. (s.f.-e). *Firewall rules in Azure Database for PostgreSQL flexible server*. Microsoft Learn. Recuperado el 30 de octubre de 2026, de https://learn.microsoft.com/en-us/azure/postgresql/flexible-server/concepts-firewall-rules

Microsoft. (s.f.-f). *Compute options in Azure Database for PostgreSQL flexible server*. Microsoft Learn. Recuperado el 30 de octubre de 2026, de https://learn.microsoft.com/en-us/azure/postgresql/flexible-server/concepts-compute

Microsoft. (s.f.-g). *Introduction to Azure Blob Storage*. Microsoft Learn. Recuperado el 30 de octubre de 2026, de https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction

Microsoft. (s.f.-h). *Azure Database for PostgreSQL flexible server pricing*. Recuperado el 30 de octubre de 2026, de https://azure.microsoft.com/en-us/pricing/details/postgresql/flexible-server/

Microsoft. (s.f.-i). *App Service pricing (Linux)*. Recuperado el 30 de octubre de 2026, de https://azure.microsoft.com/en-us/pricing/details/app-service/linux/

Microsoft. (s.f.-j). *Configure an App Service app*. Microsoft Learn. Recuperado el 30 de octubre de 2026, de https://learn.microsoft.com/en-us/azure/app-service/configure-common

Nottingham, M., & Fielding, R. (2012). *Additional HTTP status codes* (RFC 6585). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc6585

node-postgres. (s.f.-a). *Pooling*. Recuperado el 30 de octubre de 2026, de https://node-postgres.com/features/pooling

node-postgres. (s.f.-b). *Queries*. Recuperado el 30 de octubre de 2026, de https://node-postgres.com/features/queries

OpenJS Foundation. (s.f.). *Crypto | Node.js documentation*. Recuperado el 30 de octubre de 2026, de https://nodejs.org/api/crypto.html

OWASP Foundation. (2023). *OWASP API Security Top 10 – 2023*. https://owasp.org/API-Security/editions/2023/en/0x11-t10/

PostgreSQL Global Development Group. (s.f.-a). *Appendix A. PostgreSQL error codes* (PostgreSQL 16 documentation). Recuperado el 30 de octubre de 2026, de https://www.postgresql.org/docs/16/errcodes-appendix.html

PostgreSQL Global Development Group. (s.f.-b). *5.4. Constraints* (PostgreSQL 16 documentation). Recuperado el 30 de octubre de 2026, de https://www.postgresql.org/docs/16/ddl-constraints.html

Vitest. (s.f.). *Getting started*. Recuperado el 30 de octubre de 2026, de https://vitest.dev/guide/

Wilde, E. (2019). *The Sunset HTTP header field* (RFC 8594). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc8594

Zod. (s.f.). *Zod documentation*. Recuperado el 30 de octubre de 2026, de https://zod.dev/

MATERIAL COMPLEMENTARIO

Anexos

Anexo A. Declaración de uso de inteligencia artificial

El equipo declara haber usado inteligencia artificial durante el Hito 2, en los siguientes términos:

Aspecto	Detalle
Herramienta	Claude (Anthropic), modelo Claude Opus 5.5, usado a través de Claude Code en la aplicación de escritorio de Claude.
Partes del trabajo	Redacción, estructura y orden de este documento técnico de forma parcial, los diagramas (Figuras 1 y 2) y las tablas. Durante el desarrollo también se usó como apoyo para escribir y depurar código del back end, la app de escritorio, el pipeline de CI/CD, las migraciones y comprender el funcionamiento de la configuración de Azure.
Cómo se verificó	Todo el contenido del documento se contrastó con el código del repositorio (rutas, esquemas, controladores, servicios, repositorios, `db/init.sql`, migración `002` y workflow de GitHub Actions) y con la API desplegada: `/health` y `GET /driver-packages` se consultaron en vivo el 30 de octubre de 2026. Cada URL de la bibliografía se comprobó que respondiera. El equipo revisó el documento verifico cada sección y el código que describe.
Límites	Las estimaciones de carga y costo de la sección 4 son supuestos del equipo, declarados como tales. Ninguna credencial real se compartió ni aparece en este documento.

Anexo B. Catálogo de los 9 drivers reales

ID	Hardware (VEN/DEV)	Perfil	Edición	Versión	MB
1	AMD Radeon Vega (1002/1638)	Gamer	AMD Software: Adrenalin Edition WHQL	24.9.1	726
2	NVIDIA RTX 30/40 (10DE/2684)	Gamer	NVIDIA GeForce Game Ready Package Launcher	561.09	945
3	NVIDIA RTX 30/40 (10DE/2684)	Creator	NVIDIA Studio Driver Creador Package Launcher	561.09	945
4	NVIDIA RTX 30/40 (10DE/2684)	Office	NVIDIA Standard DCH Display Driver	555.85	629
5	NVIDIA RTX 30/40 (10DE/2684)	Developer	NVIDIA GeForce DCH Developer Driver	561.09	659
6	MediaTek Wi-Fi 6 MT7921 (14C3/7961)	Gamer	MediaTek Wireless LAN WHQL Driver	3.4.0.1023	9
7	Intel Iris Xe / UHD (8086/A780)	Office	Intel Processor Graphics Driver (Gen 11-14)	31.0.101.5522	745
8	Intel Wi-Fi 6/6E (8086/51F0)	Office	Intel Wireless Wi-Fi Driver (Gen 11-14)	23.40.0	72
9	Intel Bluetooth (8087/0033)	Office	Intel Wireless Bluetooth Driver (Win 10/11)	23.40.0	61

Todos con `os_environment = Win11_x64`, certificación WHQL y URL en el contenedor `drivers/` de `driwinstorage` (HTTP 200 al 30-10-2026). Fuente: `db/migrations/002_catalogo_drivers_reales.sql`.

Anexo C. Cómo levantar el servicio en otro equipo

1. Requisitos: Git 2.x, Node.js 20 o superior (Azure usa 24), npm 10 y acceso a una base PostgreSQL 16 (local o en Azure).

2. Clonar el repositorio: `git clone https://github.com/VinJimbo97/DriWinProyect.git` y entrar a `server-api`.
3. Crear la base con `db/init.sql` (sección 7.3). Para PostgreSQL local sin TLS: `PGSSLMODE=disable` y quitar `sslmode=require` de la cadena.
4. Copiar `server-api/.env.example` a `server-api/.env` y completar `DATABASE_URL` y `API_KEY` (generar con `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`).
5. Instalar y ejecutar: `npm ci`, luego `npm run dev`. Verificar con `GET http://localhost:3000/api/v1/health`.
6. Ejecutar las pruebas con `npm test` e importar en Postman la colección y el entorno de `docs/postman/`.

El README del repositorio contiene la versión completa de estas instrucciones, incluida la app de escritorio.